

BỘ MÔN KHOA HỌC DỮ LIỆU

TRÍ TUỆ NHÂN TẠO

LAB 04 – ARTIFICIAL INTELLIGENCE



TS. TRẦN NGỌC VIỆT



HỌC KỲ 1 – NĂM HỌC 2024-2025

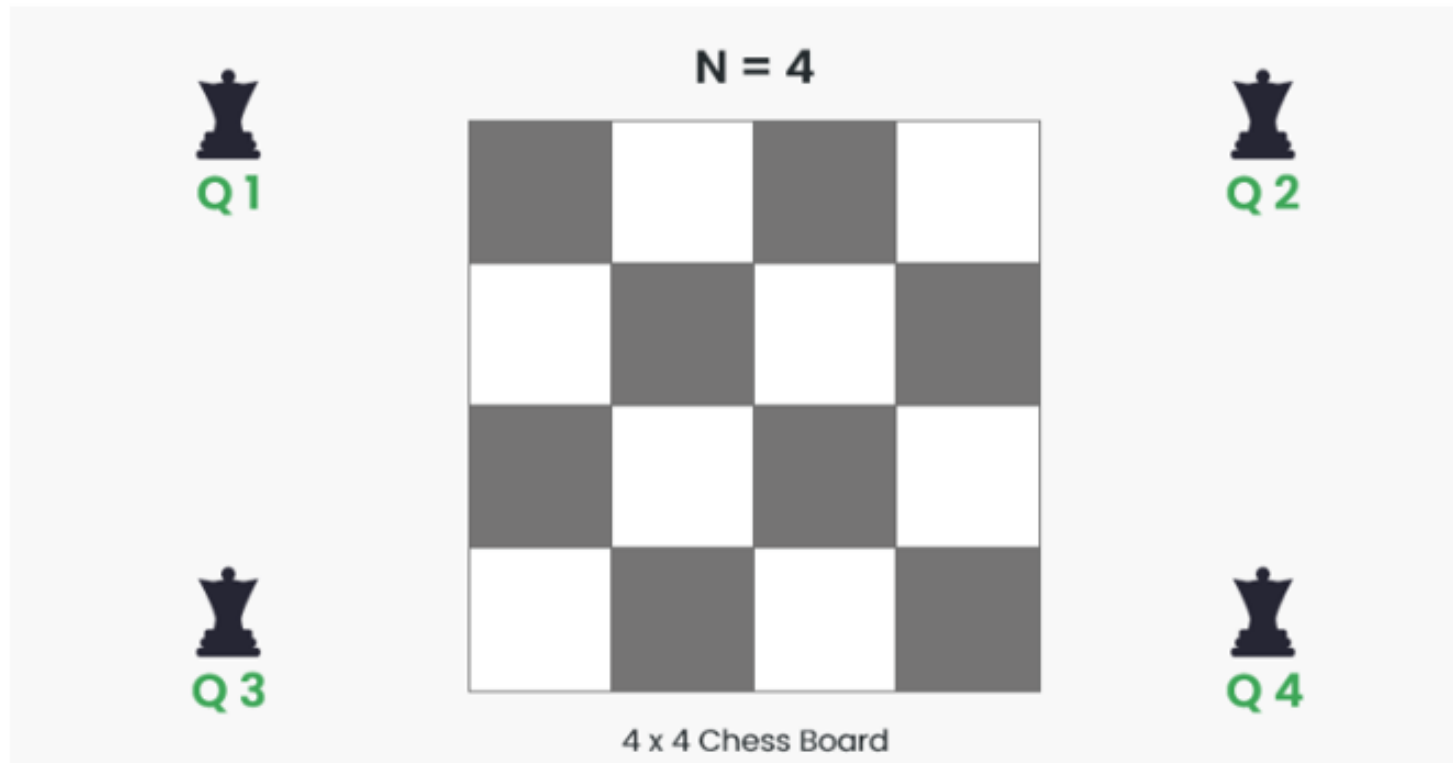


KHÓA K28 & K29

Thực hành 01 – Tìm kiếm theo ràng buộc – 4 Queens

+Bài toán $n = 4$ Queen trên một bàn cờ vua có 4×4 .

+Đặt ra không có hai Queen nào tấn công lẫn nhau. Ràng buộc điều kiện là không được phép đặt hai Queen trên cùng một hàng hoặc trên cùng một cột hoặc cùng một đường chéo.



Thực hành 01 – Tìm kiếm theo ràng buộc – 4 Queens

Step 1:

- +Đặt quân Queen Q1 vào ô (0,0).
- +‘X’ biểu thị các ô không an toàn tức là chúng đang bị Queen Q1 tấn công.
- +Sau đó di chuyển đến hàng tiếp theo.

	0	1	2	3
0	Q1	x	x	x
1	x	x		
2	x		x	
3	x			x

4 x 4 Chess Board

Step 2:

- +Đặt quân Queen Q2 tiếp theo vào ô (1, 2).
- +Di chuyển đến hàng tiếp theo.

	0	1	2	3
0	Q1	x	x	x
1	x	x	Q2	x
2	x	x	x	x
3	x		x	x

4 x 4 Chess Board

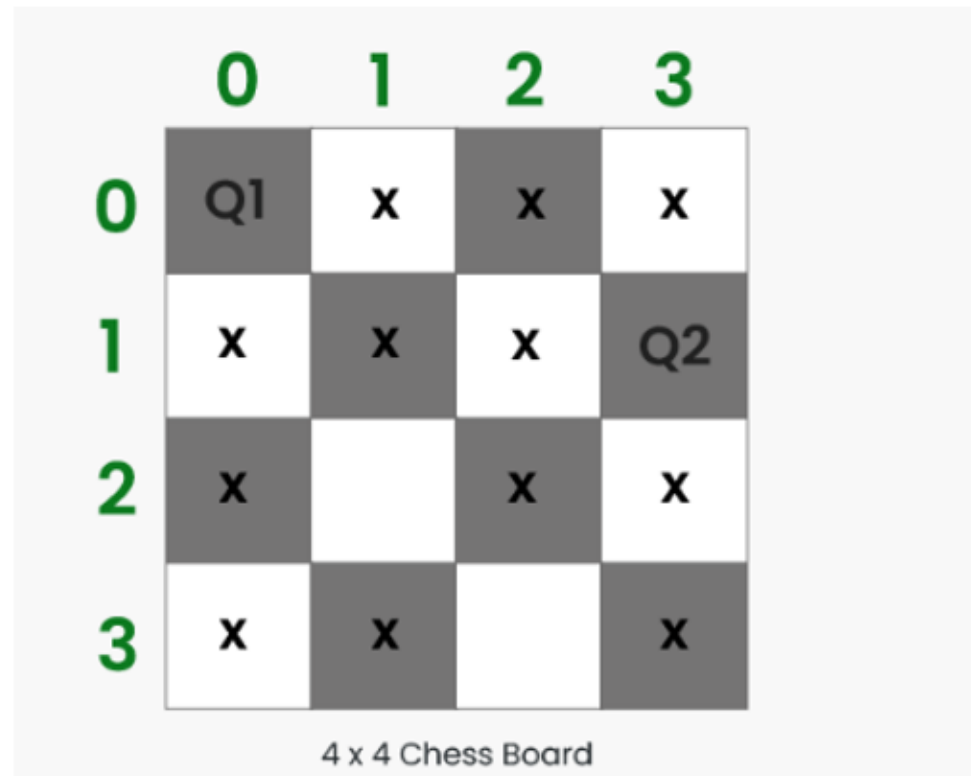
Thực hành 01 – Tìm kiếm theo ràng buộc – 4 Queens

Step 3:

- +Hàng 2 không có ô nào an toàn để đặt Queen Q3
- +Vì vậy, quay lại và loại bỏ Queen Q2 khỏi ô (1, 2)

Step 4:

- +Vẫn còn một ô an toàn ở hàng 1 tức là ô (1, 3).
- +Đặt Queen Q2 vào ô (1, 3).



Thực hành 01 – Tìm kiếm theo ràng buộc – 4 Queens

Step 5:

+Đặt Queen Q3 vào ô (2, 1)

	0	1	2	3
0	Q1	x	x	x
1	x	x	x	Q2
2	x	Q3	x	x
3	x	x	x	x

4 x 4 Chess Board

Step 6:

+Không có ô nào để đặt Queen Q4 vào hàng 3.
+Quay lại và loại Queen Q3 khỏi hàng 2.
+Tiếp tục không có ô an toàn nào khác ở hàng 2, vì vậy quay lại và loại Queen Q2 khỏi hàng 1.
+Queen Q1 sẽ bị loại khỏi ô (0,0) và di chuyển đến ô an toàn tiếp theo tức là (0, 1).

Step 7:

+Đặt Queen Q1 vào ô (0, 1) và di chuyển đến hàng tiếp theo.

	0	1	2	3
0	x	Q1	x	x
1		x	x	
2		x		x
3		x		

4 x 4 Chess Board

Thực hành 01 – Tìm kiếm theo ràng buộc – 4 Queens

Step 8:

+Đặt Queen Q2 vào ô (1, 3) và di chuyển sang hàng tiếp theo.

	0	1	2	3
0	x	Q1	x	x
1	x	x	x	Q2
2		x	x	x
3		x		x

4 x 4 Chess Board

Step 9:

+Đặt Queen Q3 vào ô (2, 0) và di chuyển sang hàng tiếp theo.

	0	1	2	3
0	x	Q1	x	x
1	x	x	x	Q2
2	Q3	x	x	x
3	x	x		x

4 x 4 Chess Board

Thực hành 01 – Tìm kiếm theo ràng buộc – 4 Queens

Step 10:

+Đặt Queen Q4 vào ô (3, 2) và di chuyển đến hàng tiếp theo.

+Kết luận: đây là giải pháp được chấp nhận.

	0	1	2	3
0	x	Q1	x	x
1	x	x	x	Q2
2	Q3	x	x	x
3	x	x	Q4	x

4 x 4 Chess Board

Thực hành 01 – Tìm kiếm theo ràng buộc – 4 Queens

```
import numpy as np
#Kiểm tra trạng thái hiện tại là giải pháp hợp lệ hay không
def is_valid_state(state, num_queens):
    return len(state) == num_queens

def get_candidates(state, num_queens):
    if not state: return range(num_queens)
    position = len(state)
    candidates = set(range(num_queens))
    for row, col in enumerate(state):
        candidates.discard(col)
        dist = position - row
        candidates.discard(col + dist)
        candidates.discard(col - dist)
    return candidates

# Recursively, perform a depth-first search to find valid solutions
def search(state, solutions, num_queens):
    if is_valid_state(state, num_queens):
        solutions.append(state.copy())
        print(f"Valid State Found: {state}")

    for candidate in get_candidates(state, num_queens):
        state.append(candidate)
        search(state, solutions, num_queens)
        print(f"backtracking from: {state}")
        state.remove(candidate)
```


Thực hành 01 – Tìm kiếm theo ràng buộc – 4 Queens

```
def solve(num_queens):  
    solutions = []  
    state = []  
    search(state, solutions, num_queens)  
    return solutions  
  
if __name__ == "__main__":  
    num_queens = int(input("Enter number of queens: "))  
    solutions = solve(num_queens)  
    for solution in solutions:  
        board = np.full((num_queens, num_queens), "-")  
        for row, col in enumerate(solution):  
            board[row][col] = 'Q'  
        print(f'\nSolution: {solution}')  
        print(board)
```

Thực hành 01 – Tìm kiếm theo ràng buộc – 4 Queens

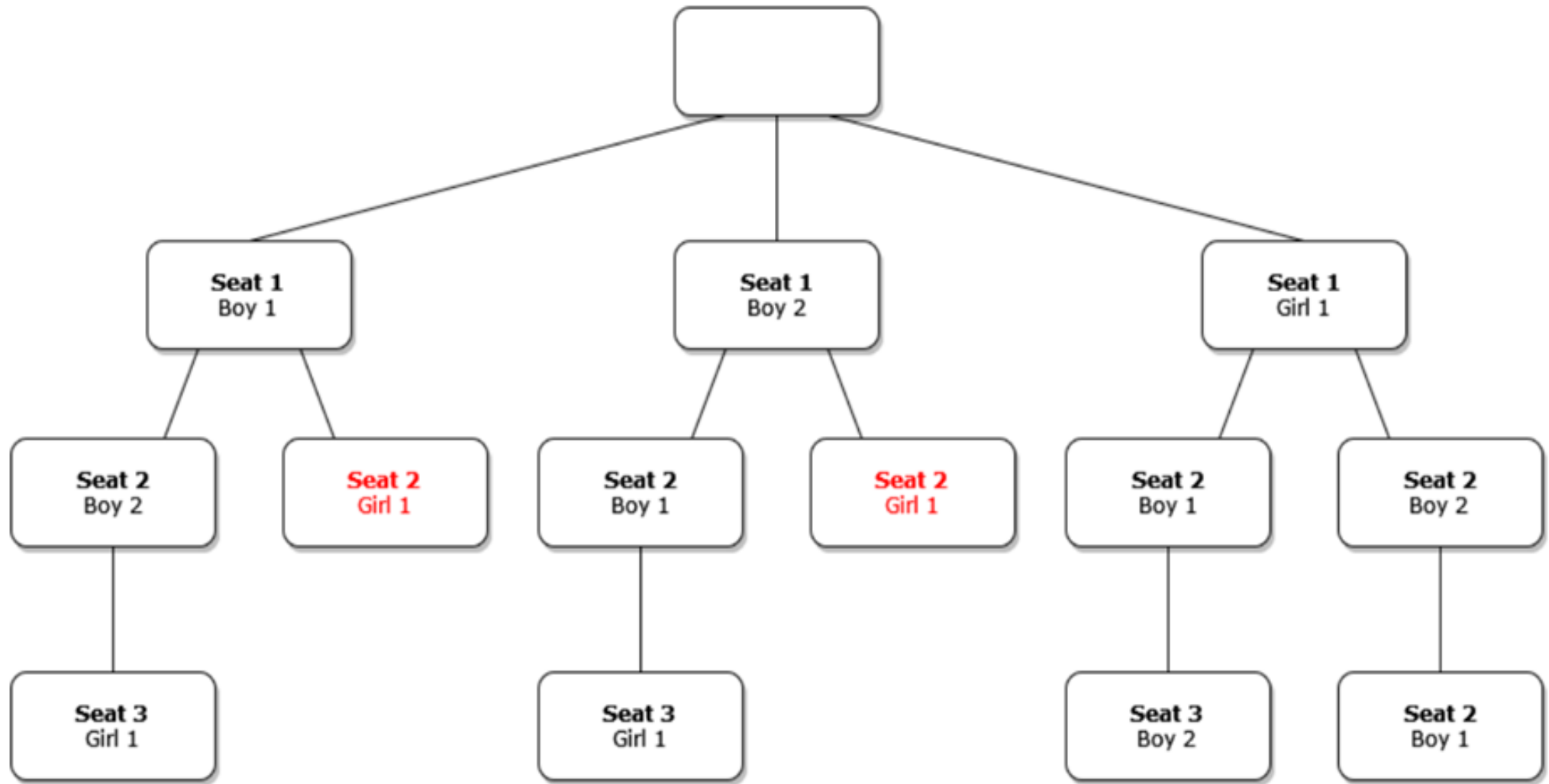
```
Enter number of queens: 4
backtracking from: [0, 2]
backtracking from: [0, 3, 1]
backtracking from: [0, 3]
backtracking from: [0]
Valid State Found: [1, 3, 0, 2]
backtracking from: [1, 3, 0, 2]
backtracking from: [1, 3, 0]
backtracking from: [1, 3]
backtracking from: [1]
Valid State Found: [2, 0, 3, 1]
backtracking from: [2, 0, 3, 1]
backtracking from: [2, 0, 3]
backtracking from: [2, 0]
backtracking from: [2]
backtracking from: [3, 0, 2]
backtracking from: [3, 0]
backtracking from: [3, 1]
backtracking from: [3]
```

```
Solution: [1, 3, 0, 2]
[['-' 'Q' '-' '-']
 ['- ' '-' '-' 'Q']
 ['Q' '-' '-' '-']
 ['- ' '-' 'Q' '-']]

Solution: [2, 0, 3, 1]
[['-' '-' 'Q' '-']
 ['Q' '-' '-' '-']
 ['- ' '-' 'Q']
 ['- ' 'Q' '-' '-']]
```

#Nhận xét: các tr/h tìm kiếm
ràng buộc

Thực hành 02 – Tìm kiếm theo ràng buộc – ràng buộc Girl không ngồi giữa



Thực hành 02 – Tìm kiếm theo ràng buộc – ràng buộc Girl không ngồi giữa

```
OPTIONS = {"B1", "B2", "G1"}

#Kiểm tra trạng thái hiện tại có phải là giải pháp hợp ko;
def is_valid_state(state):
    return len(state) == 3

def get_candidates(state):
    return list(OPTIONS.difference(set(state)))

def search(state, solutions):
    if is_valid_state(state):
        solutions.append(state.copy())
        print(f"Valid State Found: {state}")
    for candidate in get_candidates(state):
        state.append(candidate)
        search(state, solutions)
        print(f"backtracking from: {state}")
        state.remove(candidate)

def solve():
    solutions = []
    state = []
    search(state, solutions)
    return solutions

if __name__ == "__main__":
    solutions = solve()
    print(solutions)
```

Thực hành 02 – Tìm kiếm theo ràng buộc – ràng buộc Girl không ngồi giữa

```
OPTIONS = {"B1", "B2", "G1"}

#Kiểm tra trạng thái hiện tại có phải là giải pháp hợp ko;
def is_valid_state(state):
    return len(state) == 3 and state != ["B1","G1","B2"] and state != ["B2","G1","B1"]

def get_candidates(state):
    return list(OPTIONS.difference(set(state)))

def search(state, solutions):
    if is_valid_state(state):
        solutions.append(state.copy())
        print(f"Valid State Found: {state}")
    for candidate in get_candidates(state):
        state.append(candidate)
        search(state, solutions)
        print(f"backtracking from: {state}")
        state.remove(candidate)

def solve():
    solutions = []
    state = []
    search(state, solutions)
    return solutions

if __name__ == "__main__":
    solutions = solve()
    print(solutions)
```

```
Valid State Found: ['B1', 'B2', 'G1']
backtracking from: ['B1', 'B2', 'G1']
backtracking from: ['B1', 'B2']
backtracking from: ['B1', 'G1', 'B2']
backtracking from: ['B1', 'G1']
backtracking from: ['B1']
Valid State Found: ['B2', 'B1', 'G1']
backtracking from: ['B2', 'B1', 'G1']
backtracking from: ['B2', 'B1']
backtracking from: ['B2', 'G1', 'B1']
backtracking from: ['B2', 'G1']
backtracking from: ['B2']
Valid State Found: ['G1', 'B1', 'B2']
backtracking from: ['G1', 'B1', 'B2']
backtracking from: ['G1', 'B1']
Valid State Found: ['G1', 'B2', 'B1']
backtracking from: ['G1', 'B2', 'B1']
backtracking from: ['G1', 'B2']
backtracking from: ['G1']
[['B1', 'B2', 'G1'], ['B2', 'B1', 'G1'], ['G1', 'B1', 'B2'], ['G1', 'B2', 'B1']]
```

#Nhận xét: các tr/h tìm kiếm
ràng buộc

Thực hành 03 – Tìm kiếm theo ràng buộc – Solve Sudoku

```
import os
import numpy as np
from time import sleep
clear = lambda: os.system('clear')
from itertools import product

# BOARD = np.full((9,9), '.')

BOARD = np.array([
    [".", ".", "9", "7", "4", "8", ".", ".", "."],
    ["7", ".", ".", ".", ".", ".", ".", ".", "."],
    [".", "2", ".", "1", ".", "9", ".", ".", "."],
    [".", ".", "7", ".", ".", ".", "2", "4", "."],
    [".", "6", "4", ".", "1", ".", "5", "9", "."],
    [".", "9", "8", ".", ".", ".", "3", ".", "."],
    [".", ".", ".", "8", ".", "3", ".", "2", "."],
    [".", ".", ".", ".", ".", ".", ".", ".", "6"],
    [".", ".", ".", "2", "7", "5", "9", ".", "."],
])

# 9x9 dạng bảng
SHAPE = 9
# 3x3 kiểu hình vuông
GRID = 3
# vị trí trống trong bảng
EMPTY = '.'
DIGITS = set([str(num) for num in range(1, SHAPE + 1)])

def get_kth_row(board, k):
    return board[k]

def get_kth_col(board, k):
    return [board[row][k] for row in range(SHAPE)]
```

Thực hành 03 – Tìm kiếm theo ràng buộc – Solve Sudoku

```
def get_grid_at_row_col(board, row, col):
    row = row // GRID * GRID
    col = col // GRID * GRID

    return [
        board[r][c]
        # Get every [row][col] index for the sub square
        for r, c, in product(range(row, row + GRID), range(col, col + GRID))
    ]

def get_rows(board):
    for i in range(SHAPE):
        yield board[i]

def get_cols(board):
    for col in range(SHAPE):
        ret = [board[row][col] for row in range(SHAPE)]
        yield ret

def get_grids(board):
    # Iterate over each row with a stride of GRID
    for row in range(0, SHAPE, GRID):
        # Iterate over each column with a stride of GRID
        for col in range(0, SHAPE, GRID):
            grid = [
                board[r][c]
                # Get every [row][col] index for the sub square
                for r, c in product(range(row, row +
                                         GRID), range(col, col + GRID))
            ]
            yield grid
```

Thực hành 03 – Tìm kiếm theo ràng buộc – Solve Sudoku

```
#Kiểm tra trạng thái hiện tại là giải pháp hợp lệ hay ko
def is_valid_state(state):
    for row in get_rows(state):
        if set(row) != DIGITS:
            return False

    for col in get_cols(state):
        if set(col) != DIGITS:
            return False

    for grid in get_grids(state):
        if set(grid) != DIGITS:
            return False

    return True

#danh sách các bước tiếp theo
def get_candidates(state, row, col):
    used_digits = set()
    used_digits.update(get_kth_row(state, row))
    used_digits.update(get_kth_col(state, col))
    used_digits.update(get_grid_at_row_col(state, row, col))
    return DIGITS - used_digits
```


Thực hành 03 – Tìm kiếm theo ràng buộc – Solve Sudoku

```
#thực hiện tìm kiếm chuyên sâu để tìm giải pháp hợp lệ
def search(state):
    if is_valid_state(state):
        print(f"Valid State Found:\n{state}\n")
        return True

    for row_i, row in enumerate(state):
        for col_i, val in enumerate(row):
            if val == EMPTY:
                for candidate in get_candidates(state, row_i, col_i):
                    state[row_i][col_i] = candidate
                    if search(state):
                        return True
                else:
                    state[row_i][col_i] = EMPTY
            return False
    return True

def solve(board):
    search(board)

if __name__ == "__main__":

    print(f'Initial Board:\n{BOARD}\n')
    solve(BOARD.copy())
```

Thực hành 03 – Tìm kiếm theo ràng buộc – Solve Sudoku

Initial Board:

```
[['.' '.' '9' '7' '4' '8' '.' '.' '.']  
['7' '.' '.' '.' '.' '.' '.' '.' '.']  
['.' '2' '.' '1' '.' '9' '.' '.' '.']  
['.' '.' '7' '.' '.' '.' '2' '4' '.']  
['.' '6' '4' '.' '1' '.' '5' '9' '.']  
['.' '9' '8' '.' '.' '.' '3' '.' '.']  
['.' '.' '.' '8' '.' '3' '.' '2' '.']  
['.' '.' '.' '.' '.' '.' '.' '.' '6']  
['.' '.' '.' '2' '7' '5' '9' '.' '.']]
```

Valid State Found:

```
[['5' '1' '9' '7' '4' '8' '6' '3' '2']  
['7' '8' '3' '6' '5' '2' '4' '1' '9']  
['4' '2' '6' '1' '3' '9' '8' '7' '5']  
['3' '5' '7' '9' '8' '6' '2' '4' '1']  
['2' '6' '4' '3' '1' '7' '5' '9' '8']  
['1' '9' '8' '5' '2' '4' '3' '6' '7']  
['9' '7' '5' '8' '6' '3' '1' '2' '4']  
['8' '3' '2' '4' '9' '1' '7' '5' '6']  
['6' '4' '1' '2' '7' '5' '9' '8' '3']]
```

#Nhận xét: các tr/h tìm kiếm ràng buộc



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

